ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 2

BASIC PRINCIPLES OF COMPUTER PROGRAMMING AND COMPUTER LITERACY

PRACTICE EXAM 5 (PAPER 1) · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 2 Basic Principles of Computer Programming and Computer Literacy practice exam 5 (paper 1).

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 2. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept other correct answers and wording. Flowcharts may be laid out differently, as long as the correct symbols and logic are used.

SECTION A

QUESTION 1

1 1.1 C ✓ 1.2 A ✓ 1.3 B ✓ 1.4 D ✓ 1.5 A ✓ 1.6 C ✓ 1.7 B ✓ 1.8 D ✓ 1.9 A ✓ 1.10 B ✓ (10)

[10]

QUESTION 2

2 2.1 True ✓ 2.2 True ✓ 2.3 False ✓ 2.4 True ✓ 2.5 False ✓ (5)

[5]

QUESTION 3

3 3.1 D ✓ 3.2 B ✓ 3.3 G ✓ 3.4 J ✓ 3.5 H ✓ 3.6 E ✓ 3.7 C ✓ 3.8 I ✓ 3.9 F ✓ 3.10 A ✓ (10)

[10]

QUESTION 4

4 4.1 Blocks palette (block categories) ✓ 4.2 Code area (scripts area) ✓ 4.3 Stage ✓ 4.4 Sprite pane (sprite list / sprite info) ✓ 4.5 Green flag (start the program) ✓ (5)

[5]

TOTAL SECTION A: [30]

SECTION B

QUESTION 5

5.1 5.1.1 Hard disk drive / SSD ✓ 5.1.2 Connects the computer to a network ✓ 5.1.3 USB ✓ 5.1.4 Heat sink and fan (cooler) ✓ (4)

5.2 A repository is a storage place (folder) for a project's files ✓ together with the full history of all their changes ✓. A commit is a saved snapshot of the changes at a point in time ✓, with a message that describes what was changed ✓. (4)

5.3 System software runs and manages the computer and its hardware ✓ – e.g. Windows, Linux, device drivers ✓. Application software helps the user do a particular task ✓ – e.g. Word, a web browser, Scratch ✓. (4)

5.4 A syntax error breaks the rules of the language, so the program will not run ✓ – e.g. a missing colon or bracket ✓. A logic error lets the program run but gives a wrong result ✓ – e.g. using + instead of − in a calculation ✓. (4)

5.5 Any TWO, ✓✓ each: everyone can work on the same code without overwriting each other's work; all changes are recorded so mistakes can be undone / older versions restored; you can see who changed what and when; work can be merged; the code is backed up on a server such as GitHub. (4)

5.6 $37 \div 2 = 18 \text{ r } 1; 18 \div 2 = 9 \text{ r } 0; 9 \div 2 = 4 \text{ r } 1; 4 \div 2 = 2 \text{ r } 0; 2 \div 2 = 1 \text{ r } 0; 1 \div 2 = 0 \text{ r } 1$ ✓✓✓ (or $32 + 4 + 1$); reading upwards: 100101_2 ✓✓ (5)

[25]

QUESTION 6

6.1 battery ✓ starts at 20 ✓ – the battery level in %; charges ✓ starts at 0 ✓ – how many times 10% has been added (number of charging steps). (4)

6.2 The battery goes 30, 40, 50, 60, 70, 80, 90, 100 ✓✓; at 100 the condition (battery > 99) is true, so the loop stops ✓: it runs 8 times ✓. (4)

6.3 Last message in the loop: "Battery: 100" ✓✓. charges = 8, which is more than 5, so the final message is "Charging took a long time" ✓✓. (4)

6.4 A loop / iteration (repeat until) ✓ and a selection / decision (if … then … else) ✓. They control the order and flow of a program: they let it repeat steps and make decisions instead of only running line by line ✓✓. (4)

6.5 Any sensible change ✓✓✓✓, e.g. after "change battery by 10", add: if battery > 100 then set battery to 100; or change the loop to add 5 each time / to repeat until battery = 100 and only add what is still needed. (4)

[20]

QUESTION 7

7.1 Start terminator (oval) ✓ with "Start" ✓ (2)

7.2 Input / output symbol (parallelogram) ✓: "Insert money, select snack" ✓ (2)

7.3 Decision symbol (diamond) ✓ with "Money ≥ price?" ✓ and Yes ✓ / No ✓ paths labelled (4)

7.4 No path: output "Insufficient funds" ✓, return money ✓, to End ✓ (3)

7.5 Yes path: give snack ✓; change = money − price (process) ✓; give change ✓ (3)

7.6 End terminator ✓ (1)

[15]

QUESTION 8

Any TEN, one mark each ✓✓✓✓✓✓✓✓✓✓: Line 1 – missing colon after def add_vat(price); Line 2 – logic error: should be price * 1.15 to add VAT (0.15 only gives the VAT amount); Line 4 – missing closing bracket; Line 6 – logic error: range(items + 1) loops one time too many, should be range(items); Line 7 – the price is not converted to a number (float(input(…))); Line 9 – print must be indented inside the if; Line 10 – missing colon after else; Line 11 – totl is misspelt, should be total; Line 12 – a string cannot be joined to a number, use str(total) or a comma; Line 13 – Print must be lower case (print). (10)

[10]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE